

# Lingo

Language Translation Tool

Project Documentation

**CodeAlpha Internship - Task 1**

**Submitted By: Fatima Eman**

**Student ID: CA/DF1/192788**

**Full-Stack Language Translation Web App**

# 1. Project Overview

Lingo is a full-stack language translation web application built to fulfil CodeAlpha Internship Task 1: a tool where a user can enter text, select source and target languages, and receive a translated result through a translation API. The project extends the base requirement with a production-style architecture: a Flask backend, a persisted per-user history system backed by Postgres, auto language detection, text-to-speech, a fully responsive custom UI, an automated test suite, and a live public deployment.

## 1.1 Original Task Requirements

- Create a user interface where the user can enter text and select source & target languages.
- Use a translation API (e.g. Google Translate API) to process the input.
- Send the text to the API and get the translated response.
- Display the translated text clearly on the screen.
- Optional: add a copy button or text-to-speech feature for better usability.

## 1.2 What Was Built Beyond the Base Requirement

- A Flask backend, keeping translation logic server-side rather than calling a translation API directly from the browser.
- Server-persisted, per-user translation history with favorites, search, and filtering — no login system required.
- Auto language detection (60+ languages recognized, 14 selectable as translation targets).
- Text-to-speech and copy-to-clipboard, as suggested by the optional requirement.
- A fully custom, responsive UI (glassmorphism design), including a dark-mode/contrast toggle, adjustable text size, and a searchable history drawer.
- Input validation and error handling on both frontend and backend, including cross-user access protection.
- A 23-test automated test suite covering routing, validation, and error handling.
- A live public deployment on Vercel, backed by a serverless-compatible Postgres database (Neon).

## 1.3 Explicit Scope Note

This is an API-integration project, not a machine-learning research project. Translation is delegated to Google Translate (via the deep-translator library) rather than a custom-trained model — this matches the task's own instruction to "use a translation API," not to build or train one. Anywhere this document says "the translation engine," it refers to that external service, called through a clean internal interface that could be swapped for a different provider without touching the rest of the app.

# 2. Features

| Area               | Details                                                                      |
|--------------------|------------------------------------------------------------------------------|
| Language selection | Searchable dropdown with flags and native names; auto-detect source language |
| Translation engine | Google Translate (free, via deep-translator library) — no API key required   |

|                         |                                                                                                          |
|-------------------------|----------------------------------------------------------------------------------------------------------|
| Layout                  | Split view on desktop, stacks vertically on mobile                                                       |
| Feedback                | Skeleton loader while translating, toast notifications for errors/status                                 |
| Input tools             | Character counter (2000 max), word counter (CJK-aware), paste-and-clean, clear button                    |
| Output tools            | Copy to clipboard, text-to-speech (browser native, locale-matched voices)                                |
| Accessibility / display | Adjustable text size (A-/A+), contrast/dark-mode toggle — both persisted per browser                     |
| History                 | Server-persisted per-user history with favorites, search, and All/Starred filtering in a slide-in drawer |
| Swap                    | Swap source/target languages; resolves "Detect language" using the last detected language                |
| Shortcuts               | Ctrl+Enter to translate, Ctrl+K to clear, Esc to close the history drawer                                |

## 3. Technology Stack

### 3.1 Frontend

- HTML5, CSS3 (custom glassmorphism design system, no framework)
- Vanilla JavaScript (no build step, no framework dependency)
- Browser-native Web Speech API for text-to-speech
- Browser-native localStorage for the anonymous client ID, text-size, and theme preference

### 3.2 Backend

- Python 3 + Flask — REST API, serves the frontend locally/on traditional hosts
- deep-translator — free, unofficial Google Translate client
- langdetect — automatic source language detection
- psycopg2 — PostgreSQL driver
- Flask-CORS — cross-origin support
- Gunicorn — production WSGI server (used on traditional hosts)
- pytest + unittest.mock — automated test suite

### 3.3 Database & Deployment

- PostgreSQL, hosted on Neon — serverless-friendly (no local disk dependency, works from a serverless function)
- Vercel — primary deployment target; runs the backend as a Python serverless function, serves the frontend as static files from its CDN
- Docker — alternative deployment path (Back4app Containers or any Docker-compatible host)

## 4. Architecture

The project has two frontend copies and two backend entry points, by design — not duplication for its own sake, but because it supports two different hosting models from one codebase without touching the core logic.

```
lingo-translator/
├── frontend/           (source of truth for the UI)
│   ├── index.html
│   ├── css/style.css
│   └── js/script.js
├── public/            (mirror of frontend/, for Vercel's static CDN)
├── backend/
│   ├── app.py         (Flask app: all route logic lives here)
│   ├── requirements.txt
│   ├── requirements-dev.txt
│   └── tests/test_app.py (23 automated tests)
├── api/
│   └── index.py        (thin entry point — imports backend/app.py for Vercel)
├── vercel.json         (routes /api/* to the serverless function)
├── Dockerfile          (alternative: Docker-based hosts)
└── README.md
```

Request flow for a translation:

- 1. User types text and selects languages in the browser (frontend/js/script.js or public/js/script.js on Vercel).
- 2. Frontend sends a POST request to /api/translate with the text, source, and target.
- 3. Flask (running locally, on a traditional host, or as a Vercel serverless function — same code either way) validates the input, optionally auto-detects the source language, and calls Google Translate via deep-translator.
- 4. The translated text is returned as JSON and rendered in the UI.
- 5. The exchange is also saved to that browser's history via a second request to /api/history, persisted in Postgres.

### 4.1 Why Two Frontend Copies and an api/ Wrapper?

Vercel expects static assets in a conventional folder (public/) served directly by its CDN, while Flask (used for local development and traditional hosts) serves frontend/ itself. api/index.py exists purely because Vercel's Python runtime looks for a serverless function file under api/ — it does nothing but import the real Flask app from backend/app.py, so all actual logic still lives in one place.

## 5. Per-User History (No Login Required)

A key design goal was that each visitor's history should be saved for them individually — not shared with other users, and not something the developer can see. This was solved without requiring user accounts:

- On first visit, the browser generates a random unique ID using `crypto.randomUUID()`.
- That ID is stored in the browser's own `localStorage` (key: `lingo_client_id`) and persists across sessions.

- Every history request sends this ID, and the backend scopes all database reads/writes to it in Postgres.
- A user requesting another user's history or attempting to modify another user's entry receives a "not found" response — verified both manually and by an automated test (see Section 7).

Trade-off: because the ID lives in localStorage rather than a real account, history is tied to a specific browser/device. Clearing browser data or switching browsers starts a fresh history. Supporting the same history across multiple devices would require adding a real authentication system (see Section 12).

## 6. API Documentation

| Method | Path                     | Purpose                                                      |
|--------|--------------------------|--------------------------------------------------------------|
| POST   | /api/translate           | Translate text (Google Translate via deep-translator)        |
| GET    | /api/history?user_id=... | Fetch a user's saved translation history (last 50)           |
| POST   | /api/history             | Save a new translation to that user's history                |
| PATCH  | /api/history/<id>        | Toggle favorite on one history entry                         |
| DELETE | /api/history/<id>        | Delete one history entry                                     |
| GET    | /api/health              | Health check — also reports whether a database is configured |

### 6.1 Example Request — POST /api/translate

```
{
  "text": "Hello, how are you?",
  "source": "en",
  "target": "ur"
}
```

### 6.2 Example Response

```
{
  "translation": "ہیلو، آپ کیسے ہیں؟",
  "detectedLanguage": "",
  "phonetic": "",
  "alternatives": []
}
```

## 7. Automated Testing

The backend has a 23-test automated suite (backend/tests/test\_app.py), run with pytest. Every test passes in well under a second, with no internet connection and no live database required — both Google Translate and the database layer are mocked.

### 7.1 Why Mock Instead of Testing the Real Services?

The goal of this suite is to verify this project's own logic — input validation, routing, error handling, and access control — not the uptime of Google Translate or a live Postgres instance. Testing against real

external services would make the suite slow, flaky (failing when a service has a temporary hiccup unrelated to the code), and dependent on network access and live credentials just to run. Mocking keeps the tests fast, deterministic, and runnable in any environment, including CI pipelines with no secrets configured.

## 7.2 Coverage

- Input validation on every endpoint: empty text, oversized text (>2000 chars), invalid language codes, malformed JSON
- The same-source-equals-target shortcut (confirms the translation API is never called unnecessarily)
- Translation success, upstream failure (502), and empty-result handling
- Auto-detection labeling
- Full history CRUD: get, save, toggle favorite, delete
- Cross-user access protection — a user requesting another user's row receives "not found," tested directly
- Graceful, JSON-formatted errors even for unhandled exceptions (e.g. a missing DATABASE\_URL) — never Flask's default HTML error page

## 7.3 Running the Tests

```
cd backend
pip install -r requirements-dev.txt
pytest tests/ -v
```

Result: 23 passed.

## 8. Manual Error-Handling Verification

Alongside the automated suite, the following cases were manually verified against a running instance during development:

- Empty text input → 400 error, "text is required"
- Unsupported/invalid language code → 400 error, naming the unsupported code
- Malformed JSON request body → 400 error, does not crash the server
- Text exceeding 2000 characters → 400 error, rejected before calling the translation service
- Same source and target language → skips the translation call entirely, returns text unchanged
- Translation service network failure → 502 error with a friendly message; frontend shows a toast instead of failing silently
- Missing DATABASE\_URL in production → clear JSON error naming the exact problem, not a generic crash
- Text-to-speech with no matching OS voice installed → toast notification instead of silent failure

## 9. Setup & Local Installation

### 9.1 Requirements

- Python 3.9 or later
- pip
- A free Postgres database (Neon — see Section 10.1)

## 9.2 Steps (Windows / PowerShell)

```
cd backend
pip install -r requirements.txt
$env:DATABASE_URL="your-neon-connection-string"
python app.py
```

Then open `http://127.0.0.1:5000` in a browser. One server serves both the UI and the API. Without `DATABASE_URL` set, translation still works fully — only the history endpoints return a clear error instead of crashing.

## 10. Deployment

The app is live and deployed on Vercel. It's built so the same codebase can also deploy to a traditional host (Render, Back4app) with no code changes — only the hosting target differs.

### 10.1 Database: Neon (Postgres)

Neon was chosen specifically because it works from a serverless function: it's reached over the network rather than requiring a local disk. A one-time setup at [neon.tech](https://neon.tech) (no credit card) provides a connection string, set as the `DATABASE_URL` environment variable wherever the app runs.

### 10.2 Primary Host: Vercel

- Push the project to a GitHub repository (including `api/`, `public/`, and `vercel.json`).
- Import the repository on [vercel.com](https://vercel.com) (free, GitHub login, no credit card for the hobby tier).
- Vercel auto-detects the Python function in `api/` — no manual build/start command needed.
- Set the `DATABASE_URL` environment variable in the project's settings before deploying.

Why this works where an earlier SQLite-based version didn't: Vercel runs the backend as a serverless function — a fresh, temporary environment on every request, with no persistent local disk. SQLite is a file on disk, so it can't survive between requests there. Postgres (Neon) lives outside Vercel entirely, reached over the network, so it works identically whether the request came from a serverless function, a traditional server, or a laptop running the app locally.

### 10.3 Alternative Hosts (Same Code, No Changes)

| Host                | Model                            | Notes                                                                                                                    |
|---------------------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Render.com          | Traditional, always-on container | Build: <code>pip install -r backend/requirements.txt</code> · Start: <code>cd backend &amp;&amp; gunicorn app:app</code> |
| Back4app Containers | Docker                           | Uses the included Dockerfile; free tier, no credit card required                                                         |

All hosting options can point at the same Neon database, so history stays consistent regardless of which is used.

## 11. Known Limitations

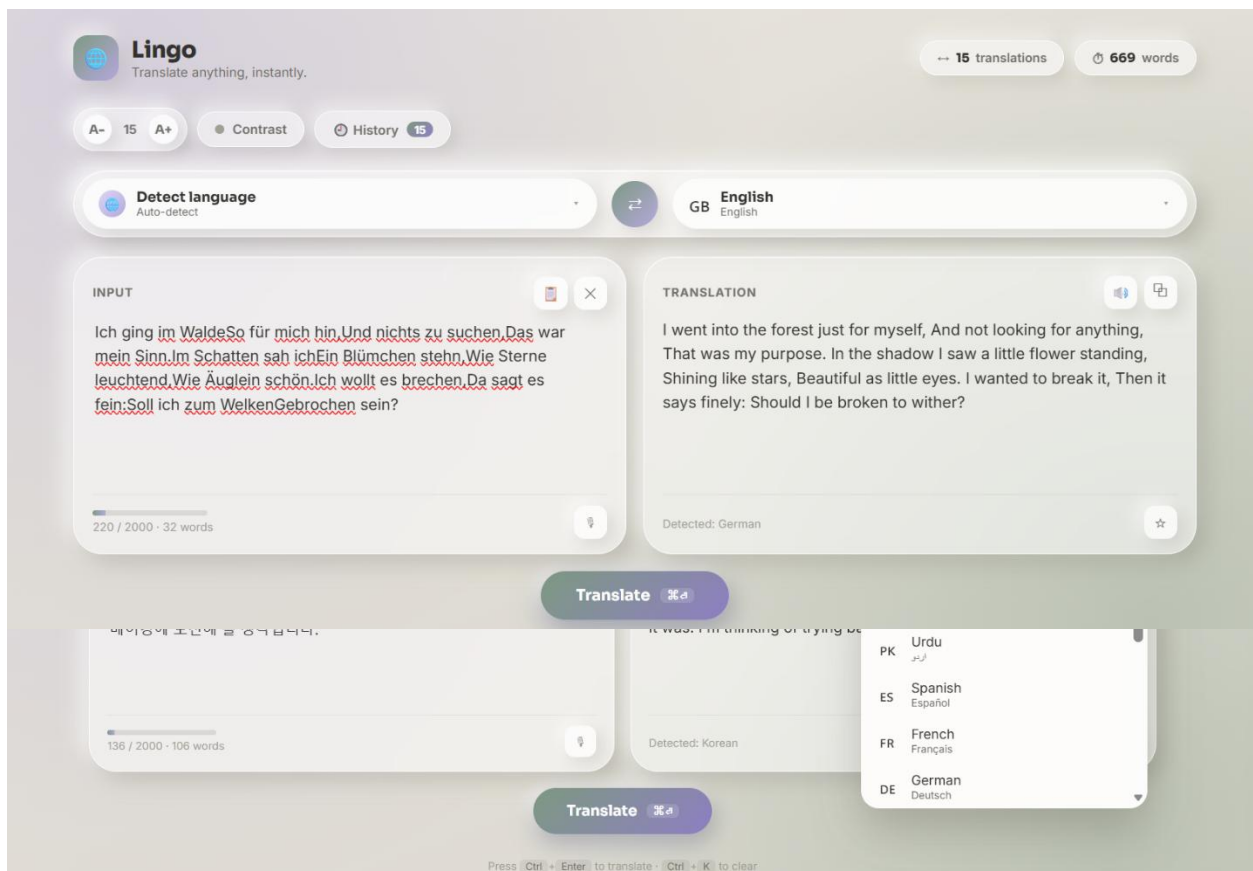
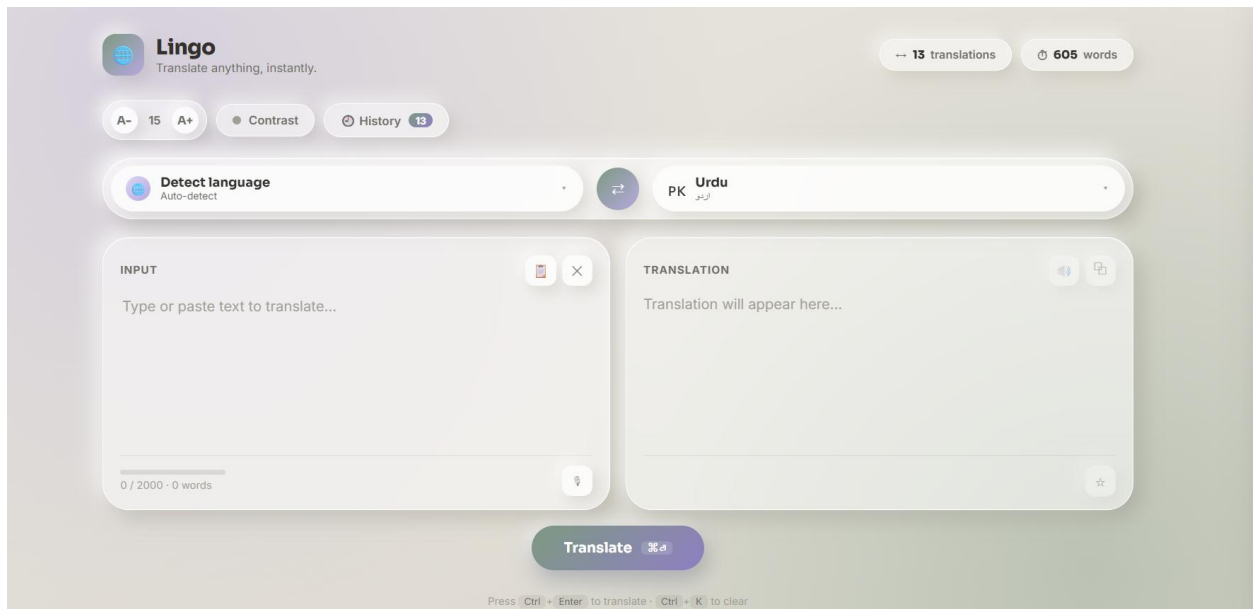
- No alternative translation suggestions or phonetic/romanized output — the free Google Translate tier does not provide these.
- Text-to-speech quality depends on voices installed on the user's OS/browser; if no matching voice exists for a language, the app shows a toast rather than failing silently — this is a genuine OS-level constraint, not fixable in application code.
- History is tied to a browser (via localStorage + a database-side client ID), not a real user account.
- Uses an unofficial, free Google Translate endpoint rather than the official paid Google Cloud Translation API — appropriate for a student project, not for commercial production traffic; no fallback provider if it's temporarily unavailable.
- Test suite covers application logic, not live third-party service behavior — by design (see Section 7.1), but worth stating explicitly.
- No rate limiting on the API — acceptable for a class project's traffic level, not for public production use without adding it.

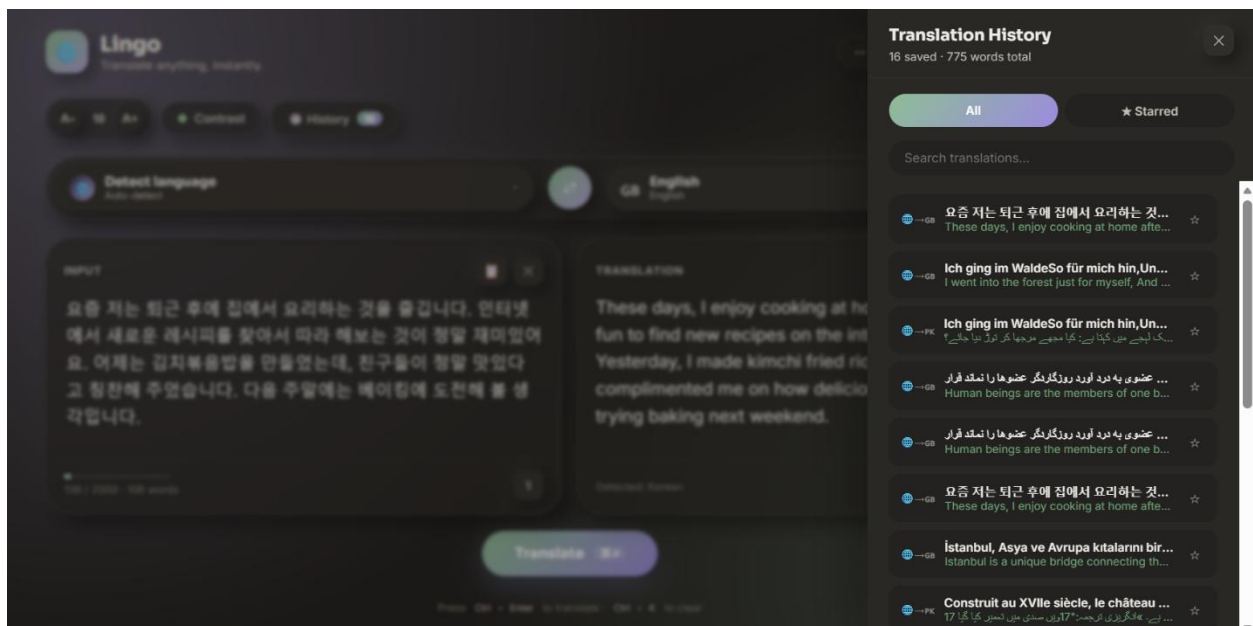
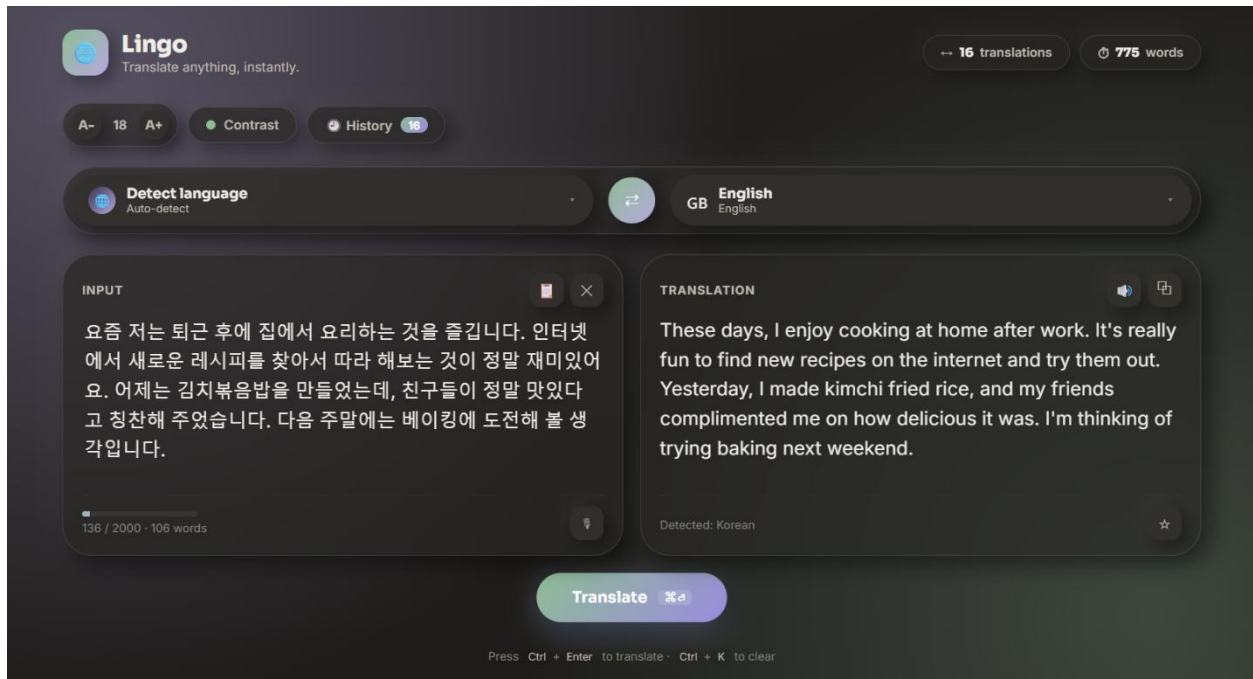
## 12. Possible Future Improvements

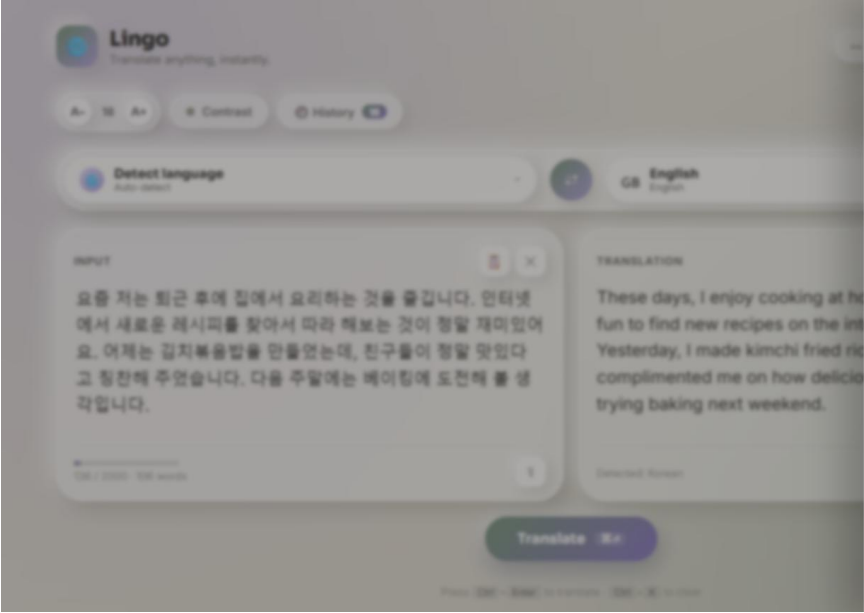
- Real user accounts (so history syncs across devices, not just one browser).
- A fallback translation provider (e.g. LibreTranslate) if Google Translate is temporarily unreachable.
- Formal/casual tone rewriting via a secondary LLM pass (attempted during development using both Google Gemini and OpenRouter; removed for this submission after Gemini's free tier turned out to be region-blocked, to keep the project dependency-free and fully working without requiring an external LLM API key).
- Rate limiting per client to protect the free translation and database tiers under heavier public traffic.
- Upgrade to the official, paid Google Cloud Translation API for guaranteed uptime and rate limits at scale.
- Expand automated tests with a live integration-test tier (running against a real disposable test database) alongside the current fast, mocked unit-test tier.

## 13. User Interface









Translation History

16 saved · 775 words total



All

★ Starred

Search translations...

Ich ging im WaldeSo für mich hin,Und ni...

I went into the forest just for myself, And not...



انچو عشوی به درد آورد روزگارنگر عضوها را نمائد قرار

Human beings are the members of one body...



انچو عشوی به درد آورد روزگارنگر عضوها را نمائد قرار

Human beings are the members of one body...

